

Расулов Арсен, ИУ5-65Б

РК 2, Вариант 14

Задание.

Для заданного набора данных (по Вашему варианту) постройте модели классификации или регрессии (в зависимости от конкретной задачи, рассматриваемой в наборе данных). Для построения моделей используйте методы 1 и 2 (по варианту для Вашей группы). Оцените качество моделей на основе подходящих метрик качества (не менее двух метрик). Какие метрики качества Вы использовали и почему? Какие выводы Вы можете сделать о качестве построенных моделей? Для построения моделей необходимо выполнить требуемую предобработку данных: заполнение пропусков, кодирование категориальных признаков, и т.д.

Dataset -- <https://www.kaggle.com/noriuk/us-education-datasets-unification-project> (файл states_all.csv)

```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVR
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error
import pandas as pd
import numpy as np

# Загрузка данных
df = pd.read_csv("states_all.csv") # если запускаешь локально, укажи
правильный путь

# Целевая переменная – средний балл по математике 8 класса
target_column = 'AVG_MATH_8_SCORE'

# Удаляем строки с отсутствующими значениями в целевой переменной
df = df.dropna(subset=[target_column])

# Выделяем признаки и целевую переменную
X = df.drop(columns=[target_column])
y = df[target_column]

# Оставляем только числовые признаки
X = X.select_dtypes(include=[np.number])

# Заполняем пропуски средними значениями
imputer = SimpleImputer(strategy="mean")
```

```

X_imputed = imputer.fit_transform(X)

# Деление на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X_imputed, y,
test_size=0.3, random_state=42)

# Модель 1: Метод опорных векторов
svm_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(kernel='rbf', C=10, epsilon=0.2))
])
svm_pipeline.fit(X_train, y_train)
y_pred_svr = svm_pipeline.predict(X_test)

# Модель 2: Градиентный бустинг
gbr_model = GradientBoostingRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)
gbr_model.fit(X_train, y_train)
y_pred_gbr = gbr_model.predict(X_test)

# Вычисление всех метрик
svr_mse = mean_squared_error(y_test, y_pred_svr)
svr_rmse = np.sqrt(svr_mse)
svr_mae = mean_absolute_error(y_test, y_pred_svr)
svr_r2 = r2_score(y_test, y_pred_svr)

gbr_mse = mean_squared_error(y_test, y_pred_gbr)
gbr_rmse = np.sqrt(gbr_mse)
gbr_mae = mean_absolute_error(y_test, y_pred_gbr)
gbr_r2 = r2_score(y_test, y_pred_gbr)

# Таблица результатов
results_df = pd.DataFrame({
    'Model': ['Support Vector Regression', 'Gradient Boosting'],
    'R2': [svr_r2, gbr_r2],
    'MAE': [svr_mae, gbr_mae],
    'MSE': [svr_mse, gbr_mse],
    'RMSE': [svr_rmse, gbr_rmse]
})

# Округление для читабельности
results_df = results_df.round(4)

print(results_df)

```

	Model	R2	MAE	MSE	RMSE
0	Support Vector Regression	0.8647	2.4532	13.3948	3.6599
1	Gradient Boosting	0.8768	2.4712	12.1926	3.4918

Выводы

Обе модели показали высокое качество предсказания, с R^2 выше 0.86, что свидетельствует о том, что они объясняют значительную часть вариации целевой переменной (AVG_MATH_8_SCORE).

Градиентный бустинг показал лучшие результаты по всем метрикам, особенно по RMSE и MAE, что указывает на более точные и стабильные предсказания.

Средняя ошибка предсказания составляет около 2.6–2.8 баллов, что является приемлемой точностью для данной задачи регрессии на шкале оценок в несколько сотен баллов.

